

پروژه پایانی درس مبانی برنامه‌نویسی پایتون

عنوان پروژه: پیاده‌سازی موتور شبیه‌ساز معاملات الگوریتمی (Algorithmic Trading Engine)

⚠️ محدودیت بسیار مهم: در این پروژه استفاده از هیچ کتابخانه‌ای به جز کتابخانه‌هایی که در طول ترم در کلاس تدریس و معرفی شده‌اند مجاز نیست. استفاده از کتابخانه‌های خارج از سرفصل درس ممنوع است.

فاز ۱: مفسر دستورات و هسته اعتبارسنجی (Validation Engine)

در این فاز، برنامه شما باید قادر باشد دستورات متنی کاربر را خوانده، تجزیه کرده و قوانین اولیه بازار را روی آن‌ها اعمال کند.

شرح وظایف

1. پارس کردن دستورات (Command Parsing)

برنامه نباید صرفاً ورودی بگیرد، بلکه باید مانند یک مفسر عمل کند. کاربر دستوراتی مانند "BUY BTC-USDT 100 54000" یا "SELL ETH-USDT 50" را وارد می‌کند. شما باید با استفاده از متدهای رشته، این دستور را به اجزای سازنده (Action, Symbol, Amount, Price) بشکنید.

ورودی: یک رشته دستور متنی از کاربر (مثلاً "BUY BTC-USDT 100 54000").

خروجی: یک ساختار داده (لیست یا تاپل) شامل چهار جزء Action, Symbol, Amount, Price، یا پیام خطای مناسب در صورت نامعتبر بودن دستور.

2. اعتبارسنجی نماد

نمادهای معاملاتی باید استاندارد باشند. تابعی بنویسید که بررسی کند نماد حتماً شامل خط تیره - باشد و بخش دوم آن الزاماً USDT باشد.

وریدی: یک رشته نماد (مثلاً "BTC-USDT").

خروجی: مقدار بولی (True/False) برای معتبر بودن نماد، یا پیام خطا برای نماد نامعتبر.

3. محاسبه کارمزد پلکانی

تابعی بنویسید که ارزش کل معامله را محاسبه کرده و کارمزد را کسر کند:

◦ اگر حجم معامله زیر ۱۰,۰۰۰,۰۰۰ تومان بود: ۰/۵٪ کسر شود.

◦ اگر حجم معامله بالای ۱۰,۰۰۰,۰۰۰ تومان بود: ۰/۲٪ کسر شود.

ورودی: مبلغ کل معامله (برحسب تومان) و در صورت نیاز مقدار و قیمت (برای محاسبه ارزش کل).

خروجی: مقدار کارمزد و مبلغ نهایی پس از کسر کارمزد (دو عدد جداگانه یا یک ساختار داده شامل هر دو).

4. تولید شناسه تراکنش (الگوریتمی)

بدون استفاده از ماژولهای آماده (مانند uuid)، تابعی بنویسید که یک شناسه

یکتای ۱۶ کاراکتری تولید کند. این شناسه باید ترکیبی از «زمان فعلی»، «نام نماد» و یک «عدد تصادفی» باشد که طبق یک الگوی ریاضی خاص (مثلاً مجموع کد اسکی کاراکترهای آن زوج باشد) تولید شود.

ورودی: حداقل رشته نماد (مثلاً "BTC-USDT") و یک عدد تصادفی؛ زمان فعلی می‌تواند درون تابع خوانده شود.

خروجی: یک رشته ۱۶ کاراکتری به‌عنوان شناسه تراکنش که شرایط یکتا بودن و الگوی ریاضی تعریف‌شده را رعایت می‌کند.

فاز ۲: دفتر کل بازار (Market Ledger) با ساختار تو در تو

در این فاز، پایگاه داده بازار را با استفاده از لیست‌های تو در تو (Nested Lists) شبیه‌سازی می‌کنید.

شرح وظایف

1. ساختار داده

بازار را به صورت یک لیست از تاپل‌ها تعریف کنید. هر تاپل نمایانگر یک ارز است و شامل تاریخچه قیمت‌ها می‌باشد:

ساختار: `Market = [(Symbol, CurrentPrice, [History_List])]`

مثال: `('BTC-USDT', 60000, [59000, 61000, 58000])`

ورودی: لیست نمادهای اولیه و قیمت‌های شروع (می‌تواند به صورت داده ثابت در برنامه تعریف شود یا از ورودی گرفته شود).

خروجی: ساختار کامل Market به شکل لیست تاپل‌های سه‌عضوی.

2. موتور نوسان‌گیری

تابعی بنویسید که بازار را به‌روزرسانی کند. برای هر ارز، یک قیمت جدید تولید کنید. قیمت قبلی باید به انتهای لیست تاریخچه (`History_List`) اضافه شود و قیمت جدید جایگزین قیمت فعلی گردد.

ورودی: ساختار فعلی Market و در صورت نیاز پارامترهای کنترل نوسان (مثلاً درصد تغییر مجاز).

خروجی: نسخه‌ی به‌روزشده‌ی Market که در آن تاریخچه هر ارز کامل‌تر و قیمت فعلی جدید ثبت شده است.

3. نمایش وضعیت بازار

با استفاده از حلقه‌های تو در تو، لیست ارزها را پیمایش کنید. برای هر ارز، علاوه بر قیمت لحظه‌ای، «میانگین قیمت در ۳ بازه زمانی آخر» را از لیست تاریخچه محاسبه و چاپ کنید.

ورودی: ساختار Market پس از چند مرحله نوسان‌گیری.

خروجی: گزارشی چاپ‌شده روی خروجی استاندارد شامل نماد، قیمت فعلی و میانگین ۳ قیمت آخر هر ارز.

4. تکنیک Unpacking

در حلقه‌های for استفاده از اندیس (`Index`) ممنوع است. باید حتماً تاپل‌های درون لیست را مستقیماً `Unpack` کنید. مثال:

for symbol, price, history in Market:

ورودی: ساختار Market.

خروجی: کد تمیزتر و خواناتر؛ از نظر منطقی خروجی خاصی تولید نمی‌کند اما در پیاده‌سازی سایر توابع این فاز استفاده می‌شود.

فاز ۲: موتور تطبیق سفارشات (Matching Engine)

این فاز پیچیده‌ترین بخش پروژه است که در آن باید دارایی کاربر و تطبیق خرید/فروش را با رعایت تغییرناپذیری تاپل‌ها مدیریت کنید.

شرح وظایف

1. کیف پول کاربر

دارایی کاربر را در یک لیست از تاپل‌ها ذخیره کنید: [(Currency, Amount)].

مثال اولیه: [('USDT', 10000), ('BTC', 0.5)]

ورودی: مقادیر اولیه موجودی هر ارز (به صورت ثابت یا ورودی کاربر).

خروجی: ساختار Wallet به شکل لیست تاپل‌های دو عضوی.

2. منطق خرید (BUY Logic)

تابعی بنویسید که نماد و مقدار خرید را دریافت کند:

◦ ابتدا در لیست Market جستجو کند تا قیمت لحظه‌ای را بیابد.

◦ سپس در لیست Wallet جستجو کند تا از موجودی کافی USDT اطمینان

حاصل کند.

◦ در صورت موفقیت، معامله را انجام دهد.

ورودی: نماد مورد نظر (مثل "BTC-USDT")، مقدار خرید، ساختارهای فعلی

Market و Wallet.

خروجی: نسخه‌ی به‌روزشده‌ی Wallet (و در صورت تمایل، ثبت یک رکورد

تراکنش جدید) یا پیام خطا در صورت کافی نبودن موجودی یا نامعتبر بودن نماد.

3. چالش تغییرناپذیری (Immutability)

از آنجایی که تاپل‌ها غیرقابل تغییر هستند، شما نمی‌توانید موجودی را مستقیماً

ویرایش کنید. باید تاپل قدیمی را از لیست حذف کرده و تاپل جدید با موجودی به روزرسانی شده را جایگزین کنید.

ورودی: لیست فعلی Wallet، نماد و مقدار جدید برای به روزرسانی.

خروجی: لیست جدید Wallet که تاپل قدیمی در آن جای خود را به تاپل اصلاح شده داده است.

4. مدیریت خطا (Robustness)

قبل از هرگونه تغییر در لیست ها، باید تمام شرایط (موجودی کافی، وجود ارز در بازار، صحیح بودن قیمت) چک شود. برنامه تحت هیچ شرایطی نباید کرش کند.

ورودی: داده های سفارش (نماد، مقدار، قیمت) به همراه ساختارهای فعلی Market و Wallet.

خروجی: در صورت خطا، پیام های خطای کنترل شده و بدون تغییر داده ها؛ در صورت صحت همه شرایط، انجام امن معامله و به روزرسانی ساختارها.

فاز ۴: تحلیلگر داده و توابع تابعی

در این فاز باید از قابلیت های پیشرفته توابع در پایتون برای گزارش گیری استفاده کنید.

شرح وظایف

1. محاسبه ارزش کل دارایی

تابعی بنویسید که با استفاده از تابع map و lambda، موجودی هر ارز در کیف پول را در قیمت لحظه ای آن (از لیست Market) ضرب کرده، لیست جدیدی از ارزش ها بسازد و مجموع آن را برگرداند.

ورودی: ساختارهای Market و Wallet.

خروجی: لیستی از ارزش هر ارز به صورت جداگانه و یک عدد نهایی به عنوان مجموع کل ارزش دارایی کاربر.

2. فیلتر فرصت های خرید

با استفاده از تابع filter، لیستی از ارزهایی را استخراج کنید که قیمت فعلی آنها نسبت به میانگین کل تاریخچه شان، بیش از ۱۰٪ افت داشته است.

ورودی: ساختار Market شامل تاریخچه کامل قیمت‌ها برای هر ارز.

خروجی: لیست نمادها یا رکوردهای بازاری که طبق این شرط «فرصت خرید» محسوب می‌شوند.

3. تحلیل تغییرات (Zip)

فرض کنید دو لیست قیمت دارید (قیمت‌های دیروز و امروز). با استفاده از تابع zip، این دو لیست را ترکیب کرده و لیستی از تاپل‌ها بسازید که تغییرات قیمت را نشان دهد:

(OldPrice, NewPrice, ChangePercent)

ورودی: دو لیست عددی هم‌طول، یکی برای قیمت‌های روز اول و دیگری برای قیمت‌های روز دوم.

خروجی: لیستی از تاپل‌ها که در هر تاپل، قیمت قدیم، قیمت جدید و درصد تغییر قیمت ذخیره شده است.

4. گزارش‌گیری پویا

تابع گزارش‌گیری باید آرگومان‌های ****kwargs** بپذیرد. برای مثال، اگر آرگومان **exclude_zeros=True** ارسال شد، ارزشایی که موجودی آن‌ها صفر است نباید در گزارش نمایش داده شوند.

ورودی: ساختارهای Market و Wallet به همراه آرگومان‌های نام‌دار (مثلاً **exclude_zeros=True**).

خروجی: یک گزارش متنی (چاپ روی صفحه یا برگرداندن به صورت رشته) مطابق تنظیمات ارسال‌شده در آرگومان‌ها.

بخش امتیاز اضافه (چالش الگوریتمی)

موضوع: ربات تحلیل تکنیکال (Moving Average Bot)

برای کسب نمره کامل تشویقی، ویژگی زیر را پیاده‌سازی کنید:

- یک ربات معامله‌گر خودکار بنویسید که یک لیست از قیمت‌های تاریخی یک ارز را دریافت کند.

- بدون استفاده از کتابخانه‌های آماده (مانند Pandas)، «میانگین متحرک ساده ۵ روزه» (SMA-5) را محاسبه کند.
- استراتژی معاملاتی: با استفاده از یک حلقه، نمودار قیمت را پیمایش کند:
 - هر جا «قیمت لحظه‌ای» خط «میانگین متحرک» را رو به بالا قطع کرد (Cross Over)، سیگنال "BUY" صادر کند.
 - هر جا رو به پایین قطع کرد، سیگنال "SELL" صادر کند.

ورودی: یک لیست عددی شامل قیمت‌های متوالی (مثلاً قیمت بسته شدن هر روز).

خروجی: فهرستی از سیگنال‌ها (مثلاً چاپ نام روز/اندیس به همراه نوع سیگنال BUY یا SELL) یا هر قالب خروجی مستندی که نقاط ورود و خروج پیشنهادی را نشان دهد.

موفق باشید.